UNIVERSITÀ DI BOLOGNA

School of Engineering

Master Degree in Automation Engineering

Optimal Control

Course Project #1 – Group 15
Optimal Control of a Vehicle

Professor: **Giuseppe Notarstefano**

Students:
Eugenio Piccini
Francesco Davide Bossio
Mirko Legnini

Academic year 2023/2024

Abstract

This project deals with the design and implementation of an optimal control law for a simple bicycle model with static load transfer. Tasks involve dynamics discretization, trajectory generation both with step and smooth change between two equilibria, trajectory tracking via LQR and MPC algorithms and at last an animation of the vehicle following the trajectory generated with the LQR algorithm.

Contents

Introduction **5**

1 Task 0 - Problem setup **6**
1.1 State space and control inputs 6
1.2 Forward Euler discretization 7

2 Task 1 - Newton's method on step trajectory **9**
2.1 Equilibrium computation 9
2.2 Optimal trajectory via Newton's algorithm 13
2.3 Conclusions 18

3 Task 2 - Newton's method on smooth trajectory **19**
3.1 Equilibrium computation and quasi-static pair 19
3.2 Newton's method on smooth curve 23
3.3 Conclusions 27

4 Task 3 - Trajectory tracking via LQR **28**
4.1 Problem setup 28
4.2 Parameters and results 28
4.3 Conclusions 36

5 Task 4 - Trajectory tracking via MPC **37**
5.1 Problem setup 37
5.2 Parameters 37
5.3 Results 38
5.4 Conclusions 38

6 Task 5 - Animation **46**

Conclusions **48**

Introduction

In this project, we are required to design an optimal trajectory, so a sequence of state-input pairs ($\mathbf{x}$,$\mathbf{u}$) that satisfies the system dynamics and minimize a designed cost function, for a vehicle modeled by a bicycle model.
In order to achieve this goal we will work on four different tasks:

- Task 0 - Problem setup: Discretize and find the expression for the gradients of the system dynamics and code them.
- Task 1 - Step trajectory generation: Compute two equilibria of the system and define a reference (step) curve between the two. Compute the optimal transition between the two exploiting the Newton's-like algorithm for optimal control.
- Task 2 - Smooth trajectory generation: Use the same algorithm as above to track a smooth reference curve.
- Task 3 - Trajectory tracking via LQR: Linearize the model around the optimal trajectory obtained before and define the optimal feedback controller to perform trajectory tracking via LQR algorithm.
- Task 4 - Trajectory tracking via MPC: Perform trajectory tracking now using the MPC algorithm.
- Task 5 - Animation: Produce a simple animation of the vehicle executing Task 3.

Contributions

Chapter 1

Task 0 - Problem setup

The aim of this task is to discretize the system dynamics, compute the gradients of the functions and code them.

1.1 State space and control inputs

The system can be modeled as a simple bicycle model, as shown in Figure 1.1.

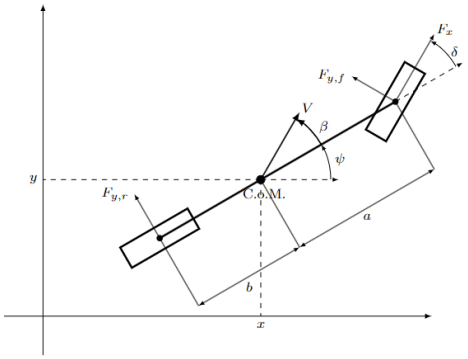


Figure 1.1: Bicycle model

The state space variables are

$$x = [x, y, \psi, V, \beta, \dot{\psi}]^T$$

where: x and y represent the position of the center of mass [CoM], ψ the angle between the horizontal and the chassis, V the velocity of the CoM with respect to the inertial frame, β the angle between the chassis and the velocity vector, $\dot{\psi}$ the angular rate of changes associated to ψ. The input is

$$u = [\delta, F_x]^T$$

where δ is the steering angle and F_x is the force applied by the wheels.

The dynamic model is described by:

$$\begin{cases} \dot{x} & = V\cos\beta\cos\psi - V\sin\beta\sin\psi \\ \dot{y} & = V\cos\beta\sin\psi + V\sin\beta\cos\psi \\ m\dot{V} & = F_{y,r}\sin\beta + F_x\cos(\beta-\delta) + F_{y,f}\sin(\beta-\delta) \\ \dot{\beta} & = \frac{1}{mV}(F_{y,r}\cos\beta + F_{y,f}\cos(\beta-\delta) - F_x\sin(\beta-\delta)) - \dot{\psi} \\ I_z\ddot{\psi} & = (F_x\sin\delta + F_{y,f}\cos\delta)a - F_{y,r}b \end{cases}$$

The lateral forces $F_{y,f}$ and $F_{y,r}$ can be found as:

$$F_{y,f} = \mu F_{z,f}\beta_f$$
$$F_{y,r} = \mu F_{z,r}\beta_r$$

where β_f and β_r are the front and rear sideslip angles:

$$\beta_f = \delta - \frac{V\sin\beta + a\dot{\psi}}{V\cos\beta}$$
$$\beta_r = -\frac{V\sin\beta - b\dot{\psi}}{V\cos\beta}$$

The vertical forces on the front and rear wheel can be found as:

$$F_{z,f} = \frac{mgb}{a+b}$$
$$F_{z,r} = \frac{mga}{a+b}$$

The mechanical parameters of the car are:

m: mass of the car	1480	[Kg]
I_z: inertia of the car	1950	$[kgm^2]$
a: distance between rear wheel and CoM	1.421	[m]
b: distance between front wheel and CoM	1.029	[m]
μ: friction coefficient	1	[nodim]
g: acceleration due to gravity	9.81	$[m/s^2]$

1.2 Forward Euler discretization

In order to write the discrete-time state-space equations, we consider the discrete time version of the bicycle model dynamics, discretized via forward Euler:

$$x_{t+1} = x_t + \delta f_{CT}(x_t, u_t, t)$$

where $\delta > 0$ is a sufficiently small discretization step and $f_{CT}(\cdot,\cdot,\cdot)$ is the continuous-time dynamics. In particular, we choose $\delta = 10^{-2}$.

We obtain the following discrete-time state-space equations:

$$\begin{cases} x_{t+1} &= x_t + \delta(V\cos\beta\cos\psi - V\sin\beta\sin\psi) \\ y_{t+1} &= y_t + \delta(V\cos\beta\sin\psi + V\sin\beta\cos\psi) \\ \psi_{t+1} &= \psi_t + \delta(\dot{\psi}) \\ V_{t+1} &= v_t + \delta\left(\frac{1}{m}\left(F_{y,r}\sin\beta + F_x\cos(\beta-\delta) + F_{y,f}\sin(\beta-\delta)\right)\right) \\ \beta_{t+1} &= \beta_t + \delta\left(\frac{1}{mV}\left(F_{y,r}\cos\beta + F_{y,f}\cos(\beta-\delta) - F_x\sin(\beta-\delta)\right) - (\dot{\psi})\right) \\ \dot{\psi}_{t+1} &= \dot{\psi}_t + \delta\left(\frac{1}{I_z}\left((F_x\sin\delta + F_{y,f}\cos\delta)a - F_{y,r}b\right)\right) \end{cases}$$

Next, since we are dealing with a non-linear system that is time-invariant, it is necessary to linearize the model by taking its gradient with respect to the state, which is defined as the transpose of the Jacobian Matrix:

$$\nabla_1 f(x,u) = [\nabla_1 f_1(x,u), \nabla_1 f_2(x,u), \nabla_1 f_3(x,u), \nabla_1 f_4(x,u), \nabla_1 f_5(x,u), \nabla_1 f_6(x,u)]$$

After that, we compute the gradient of our dynamics with respect to our input:

$$\nabla_2 f(x,u) = [\nabla_2 f_1(x,u), \nabla_2 f_2(x,u), \nabla_2 f_3(x,u), \nabla_2 f_4(x,u), \nabla_2 f_5(x,u), \nabla_2 f_6(x,u)]$$

In principle, in order to implement a precise Newton's method, we would also need to compute the Hessian of the dynamics. However, since we used the regularized Newton's method, we don't need these computation as only the hessians of the costs will be needed. We will discuss other implications for this choice in the next chapter.

Chapter 2

Task 1 - Newton's method on step trajectory

The aim of this task is to compute a desired input state curve and create an optimal trajectory that minimizes the cost with respect to this desired curve.

2.1 Equilibrium computation

The desired curve is required to start and end in equilibrium points for the system. We don't consider the trivial equilibrium for a set position, namely

$$f_1(x_t, u_t) = 0$$
$$f_2(x_t, u_t) = 0$$
$$f_3(x_t, u_t) = 0$$

Which implies $x_4 = x_5 = x_6 = 0$, as they wouldn't give us an insightful understanding of the system behavior. We instead considered the so called cornering equilibria of the system, for which the following holds:

$$f_1(x_t, u_t) \neq 0$$
$$f_2(x_t, u_t) \neq 0$$
$$f_3(x_t, u_t) \neq 0$$
$$f_4(x_t, u_t) = 0$$
$$f_5(x_t, u_t) = 0$$
$$f_6(x_t, u_t) = 0$$

This trajectories can be characterized as either a uniform linear or a uniform circular movement. Since the states x_1, x_2, x_3 do not have a role in the dynamics for the last three states, as they are just integrated from the latter, we can find these cornering equilibria by studying the reduced system

$$m\dot{x_4} = F_{y,r}\sin x_5 + u_2\cos(x_5 - u_1) + F_{y,f}\sin(x_5 - u_1)$$
$$\dot{x_5} = \frac{1}{mx_4}(F_{y,r}\cos x_5 + F_{y,f}\cos(x_5 - u_1) - u_2\sin(x_5 - u_1)) - x_6$$
$$I_z\dot{x_6} = (u_1\sin u_1 + F_{y,f}\cos u_1)a - F_{y,r}b$$

Which is a three equation system in five variables. This means we have two degrees of freedom in choosing the desired equilibrium variables, and find the others by solving the system. We decided to set the velocity module x_4 and the velocity angle x_5. We've found the other equilibrium values for the system with a zero finding method, using `sp.fsolve` from the scipy toolbox.
By imposing $(x_4^1, x_5^1) = (8, 0)$ and $(x_4^1, x_5^1) = (12, \frac{\pi}{6})$ we have found the cornering equilibrium pairs for the system dynamics.
We proceeded by integrating the dynamics, which is useful both to check the values and to create the desired curve. The complete desired curve is computed by concatenating the two subtrajectories.

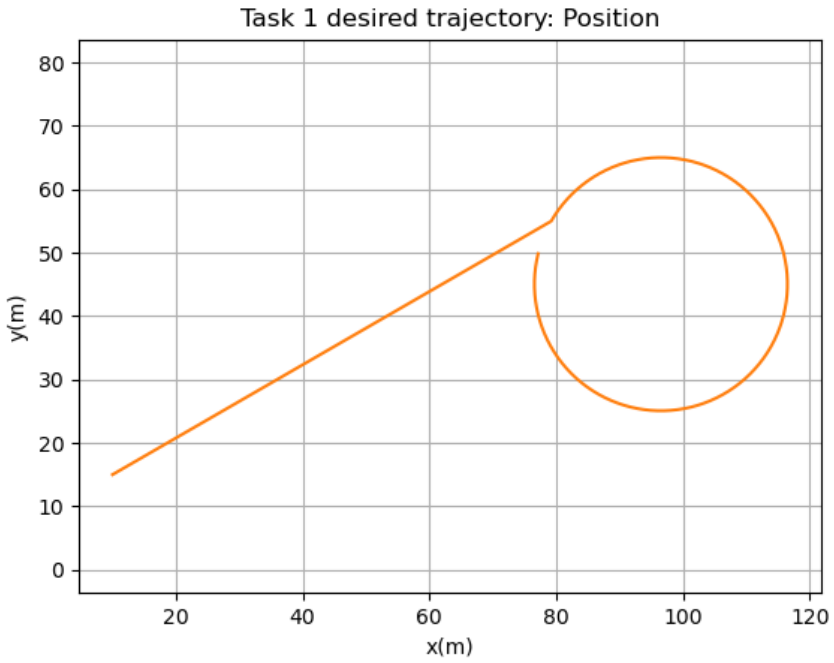


Figure 2.1: Desired step position trajectory

Figure 2.2: Desired step ψ trajectory

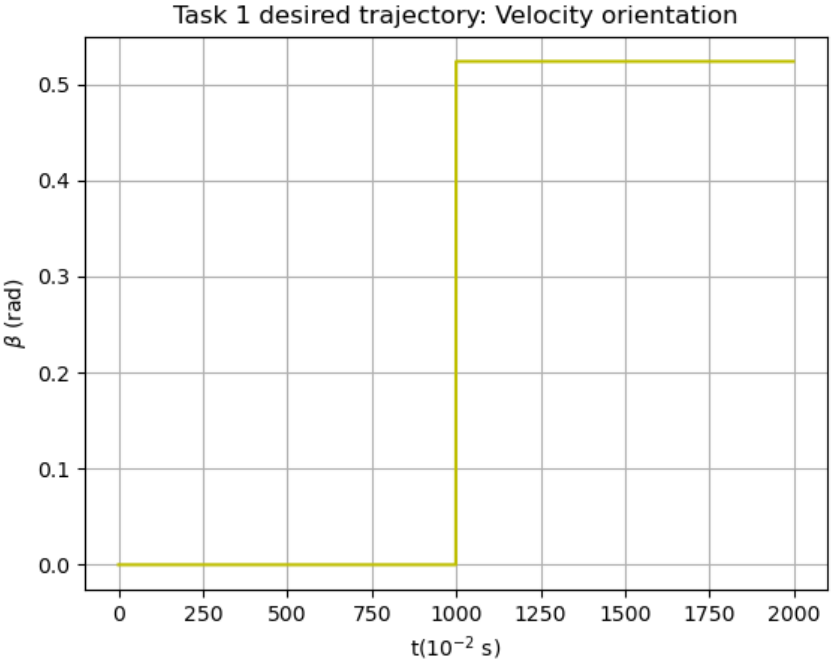


Figure 2.3: Desired step β trajectory

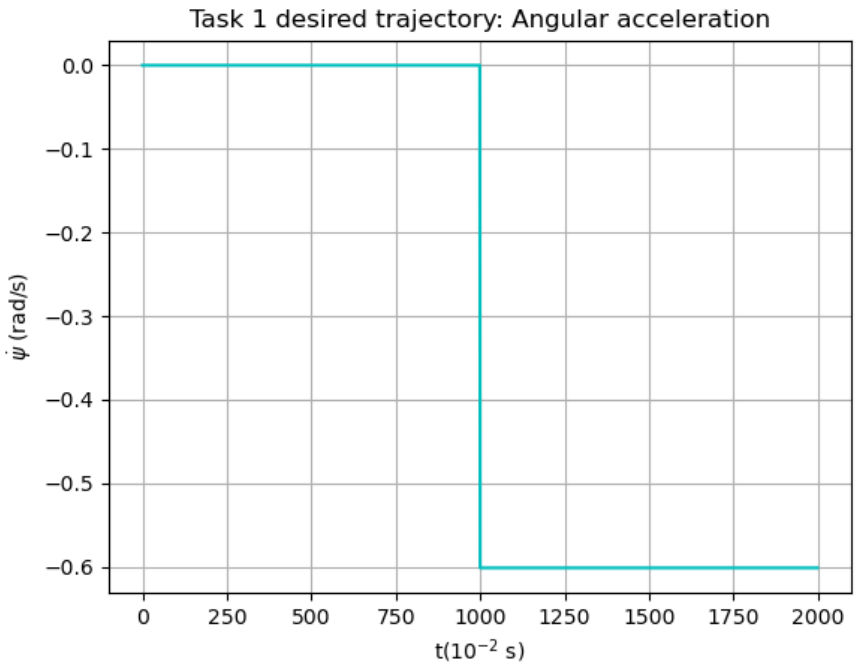


Figure 2.4: Desired step $\dot{\psi}$ trajectory

2.2 Optimal trajectory via Newton's algorithm

To find an optimal trajectory between the two equilibria we will use a cost function that involves the concept of quadratic form that allows us to input as parameters two weight matrices that represents how much we want to track accurately a variable. The first weight matrix `QQt = 0.01*diag(0.1, 0.1, 100, 10, 100, 100)` is used for the state variables while the second one `RRt = 0.01*diag(5000, 0.00001)` is for the input variables.
This cost function will be minimized via a Newton's method, for the computation of the descent direction, in which we can avoid to compute the Hessian of the dynamics by solving the costate equations and solving an affine linear quadratic problem; and with a step size (γ) chosen by the Armijo's rule.
Additionally by using the shooting method we can further simplify the problem and consider it as unconstrained optimization problem with a single decision variable u.
To sum up the decision variable u will be updated as:

$$u_t^{k+1} = u_t^k + \gamma^k \Delta u_t^k$$
$$x_t^{k+1} = f_t(x_t^{k+1}, u_t^{k+1})$$

The plots for which we need to pay more attention are the Armijo's one, in fact is possible to see if the algorithm is properly working, if we are moving in the correct direction and if the step size chosen is the correct one or is outside of the admissible area between the green and red lines. The following plots are taken every two iteration of the code.

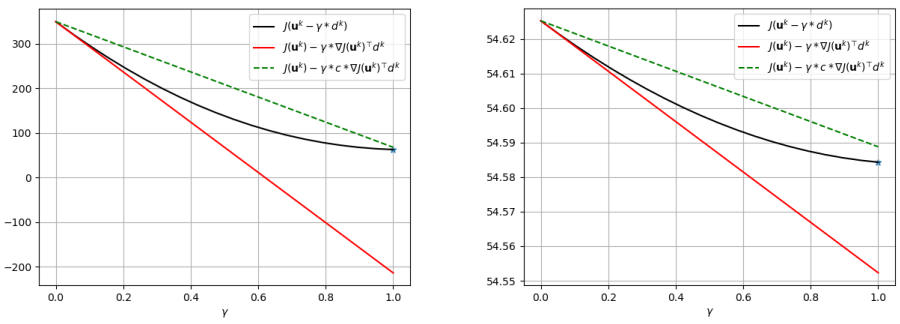


Figure 2.5: Armijo's plot at the first and third iteration

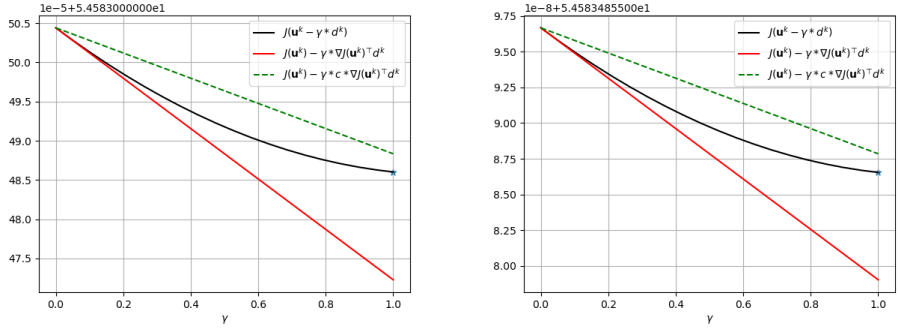


Figure 2.6: Armijo's plot at the fifth and seventh iteration

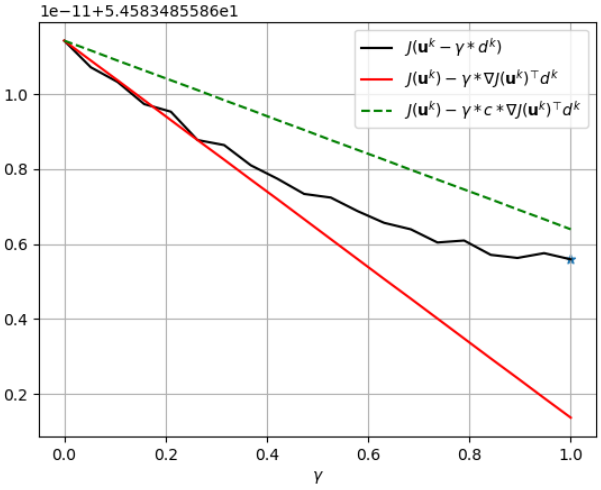


Figure 2.7: Armijo's plot at the last iteration

As we can see from figure 2.7 the algorithm is getting very close to the chosen sample period of the discretization and the result is that the curves are no longer so exact as in the first iterations so the central curve is no longer tangent to the red line.
In the following we can se the result of the applied methods on the obtained trajectories of the states and the inputs applied to the system.

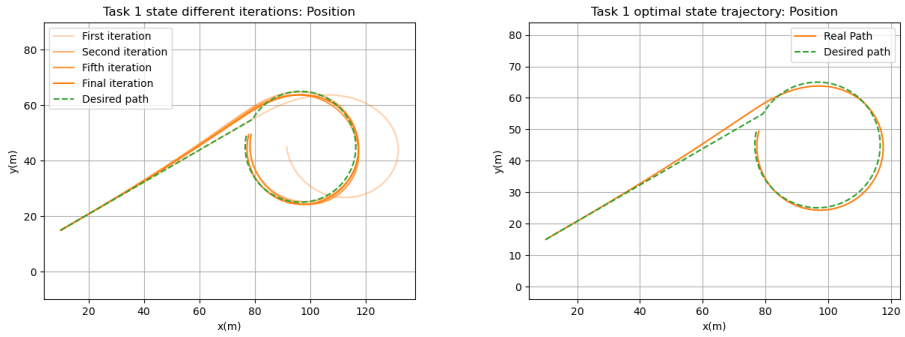


Figure 2.8: Desired and obtained step position trajectory

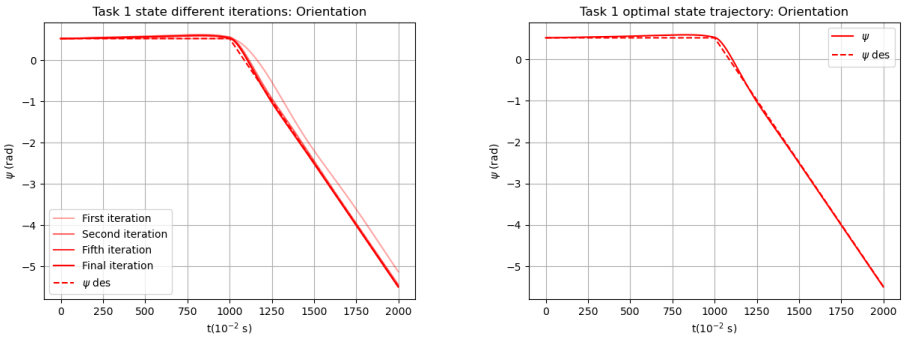


Figure 2.9: Desired and obtained step orientation (ψ) trajectory

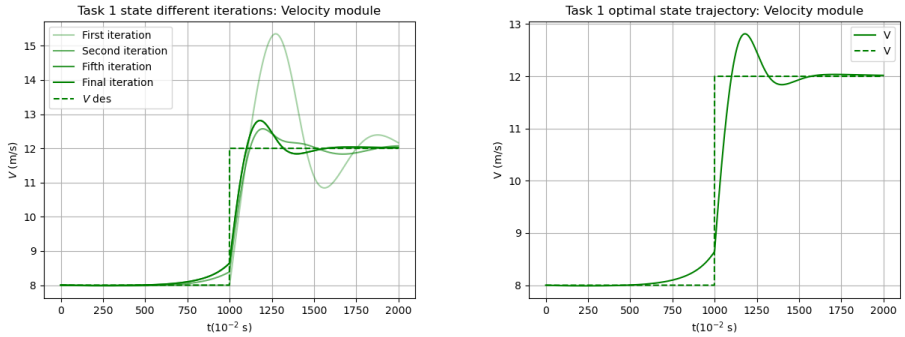


Figure 2.10: Desired and obtained step velocity trajectory

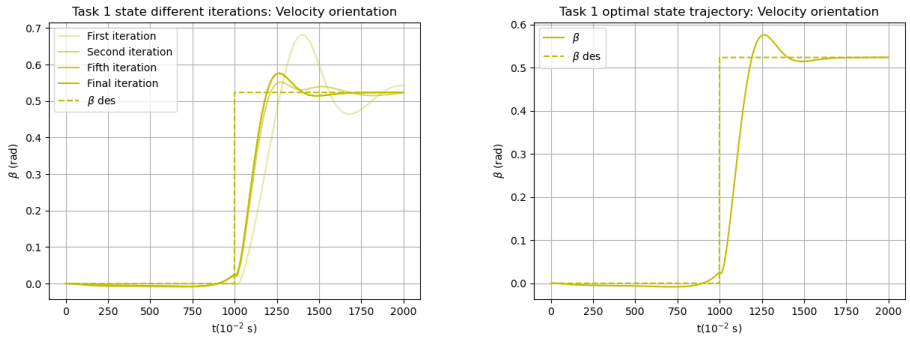


Figure 2.11: Desired and obtained step β trajectory

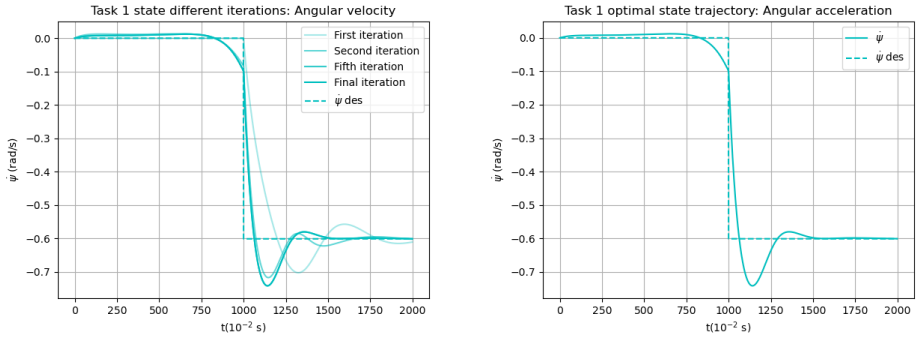


Figure 2.12: Desired and obtained step $\dot{\psi}$ trajectory

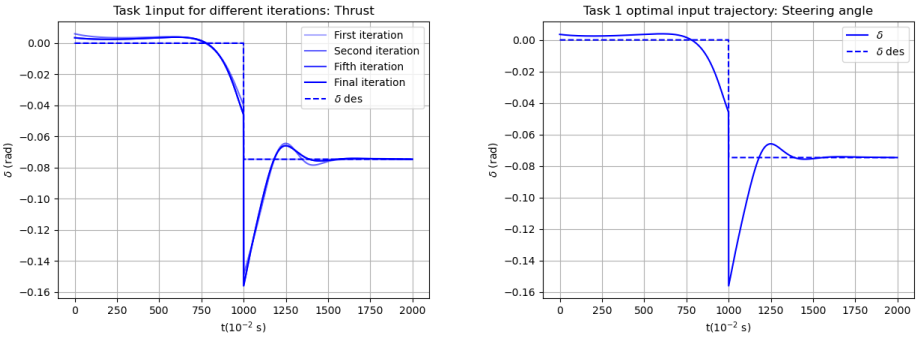


Figure 2.13: Desired and obtained step δ trajectory

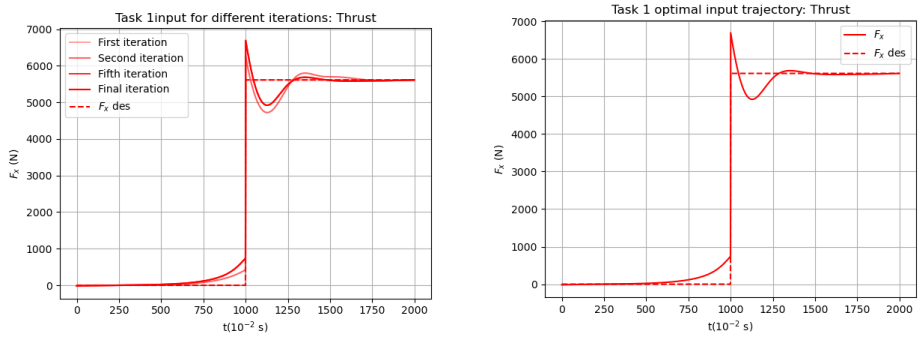


Figure 2.14: Desired and obtained step F_x trajectory

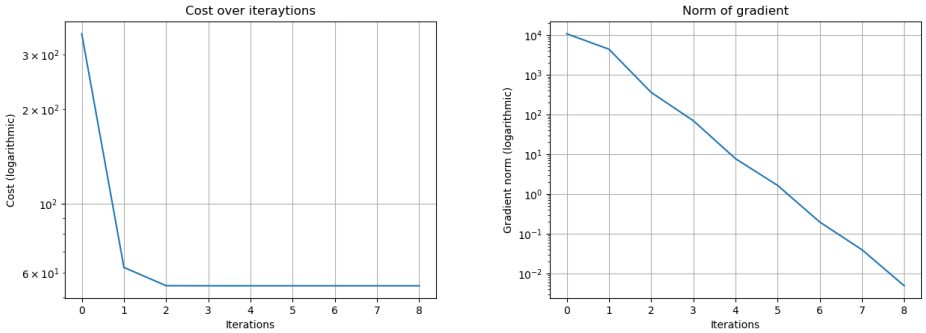


Figure 2.15: Obtained decrease in cost function and norm of descent direction

2.3 Conclusions

As we can see from figures 2.13 and 2.14 the request of a step change between the two equilibria causes some overshoot in the control inputs in particular in δ, however both the control inputs then stabilize at the desired values. We found the values of the input cost matrix `RRt` to be extremely relevant: since the force F_x is five orders of magnitude greater than the other values, we had to take a very small penalty to that term in order to balance it.
We assumed this is due to the fact that the Newton's method uses a second order approximation and this outlier input value could make the higher order terms more relevant in the computations.

Chapter 3

Task 2 - Newton's method on smooth trajectory

The second task required us to compute a smooth input state curve and find the optimal trajectory with respect to that.

3.1 Equilibrium computation and quasi-static pair

The first choice to be made is which smooth curve to use to move between the two equilibrium values. In order to attain a smooth and at least twice derivable curve, namely $x(\cdot) \in C^2$, we decided to use a sigmoid. We tuned the parameters to make the curve symmetric with respect to the positive time interval, and to have a slower transition between the values. The desired curve we obtained for β and V ended up like:

$$x(t) = x^1_{eq} + (x^2_{eq} - x^1_{eq})\sigma(\frac{(t - \frac{T}{2})}{4T}) \tag{3.1}$$

We then obtain the desired curve for the restricted system by finding the quasi-static equilibrium pair for each of the points we found, as previously done in Task 1. This way we end up with the sigmoid desired curve for the velocities. In order to find the desired positions, we integrate them by using the velocities we just found. The final result can be found in figure:

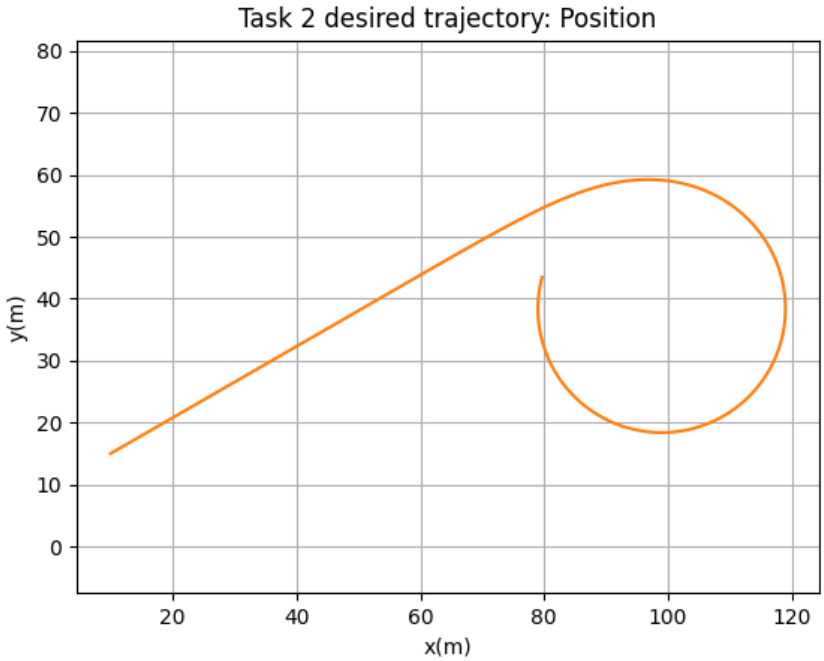


Figure 3.1: Desired smooth position trajectory

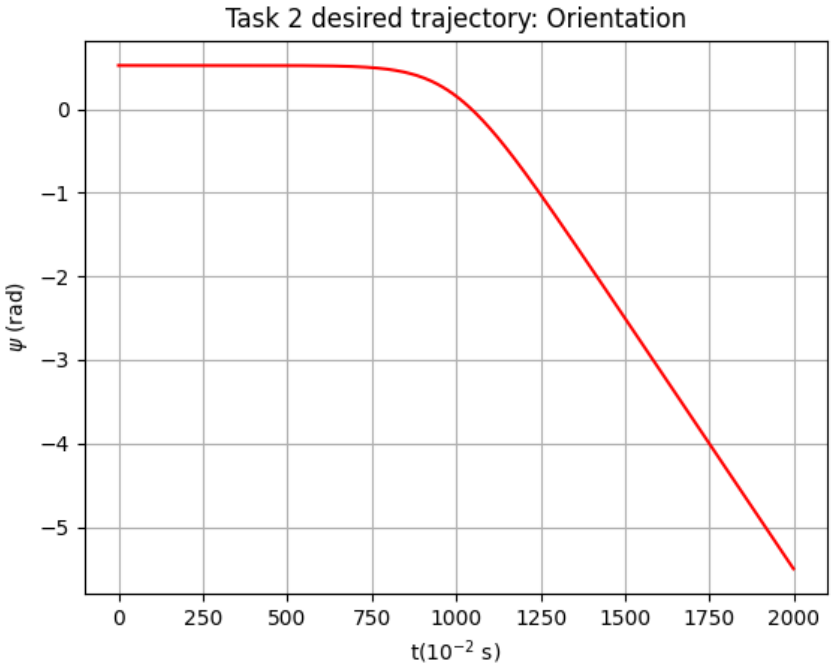


Figure 3.2: Desired smooth ψ trajectory

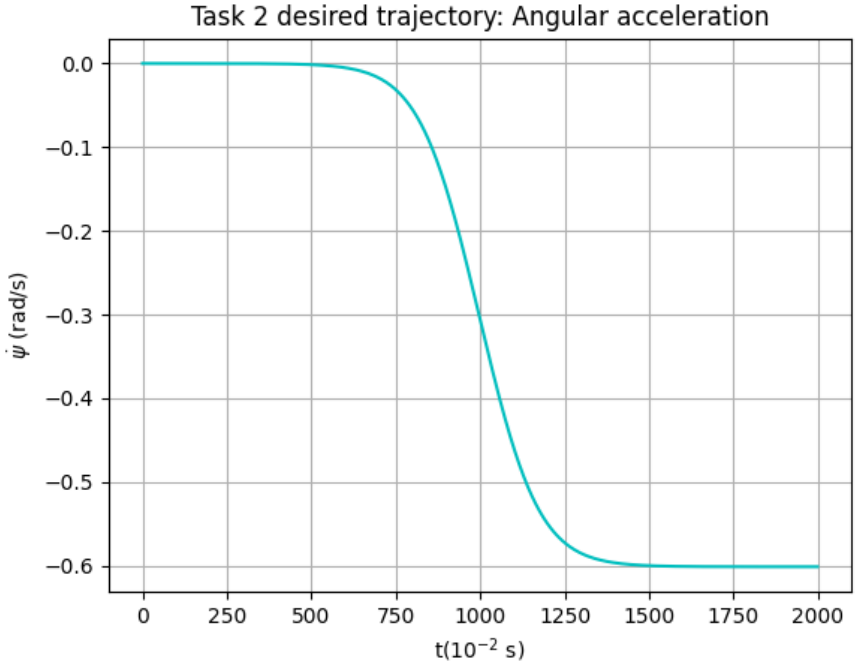


Figure 3.3: Desired smooth $\dot{\psi}$ trajectory

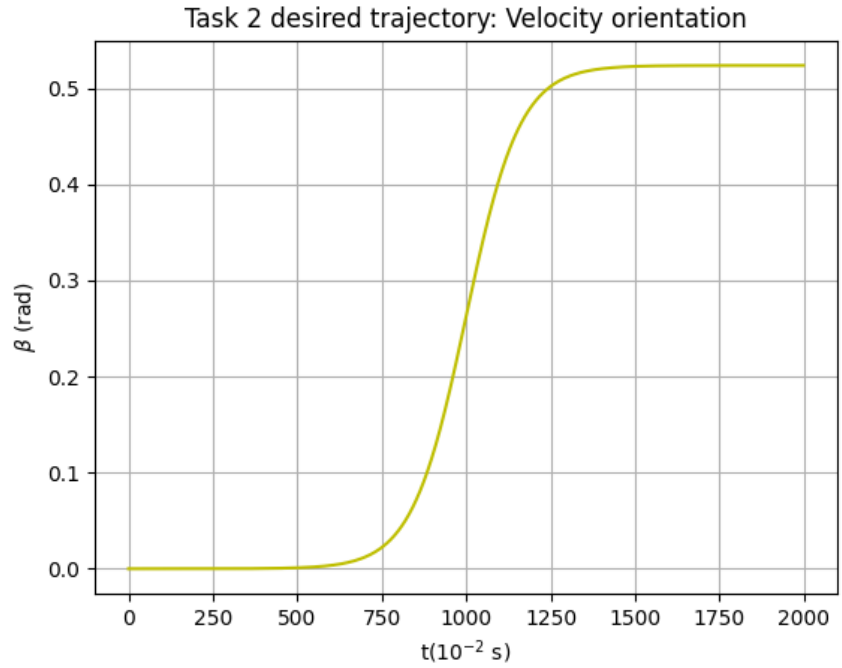


Figure 3.4: Desired smooth β trajectory

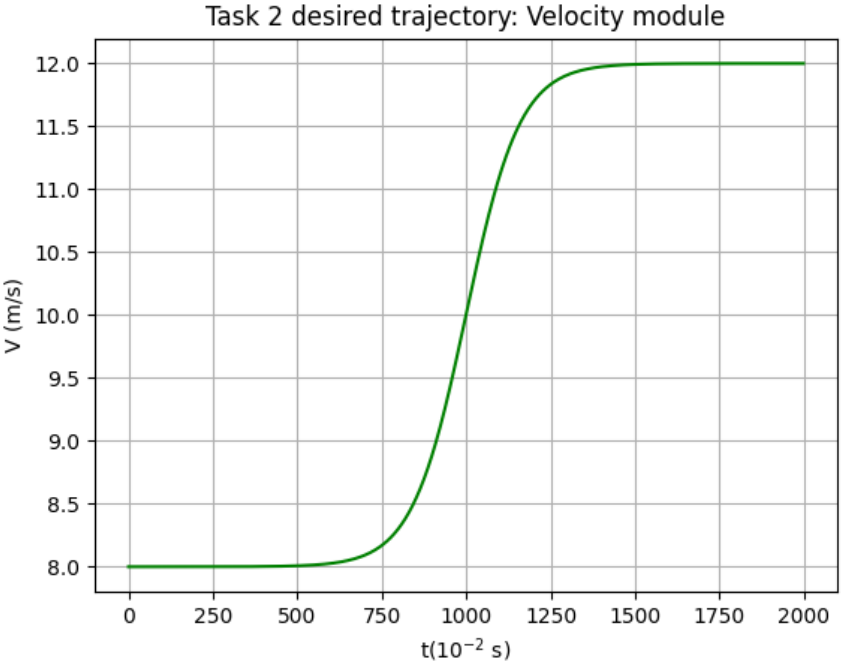


Figure 3.5: Desired smooth V trajectory

3.2 Newton's method on smooth curve

We proceded with the Newton's method much in the same way we did for task one. We used the same cost matrices and applied the same routine. In figure the results at different iterations can be found.

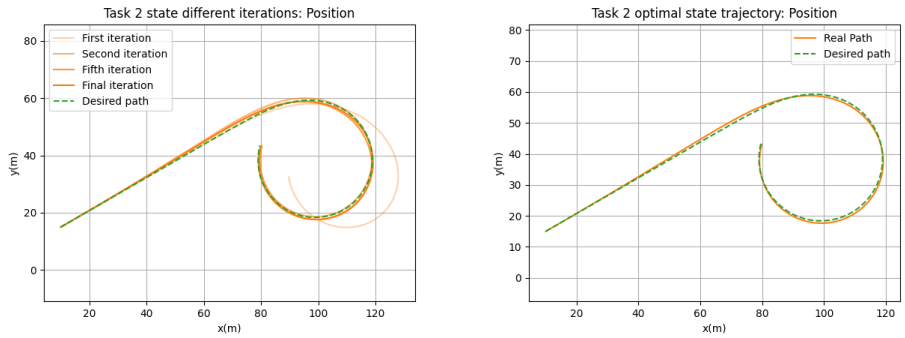


Figure 3.6: Desired smooth position trajectory

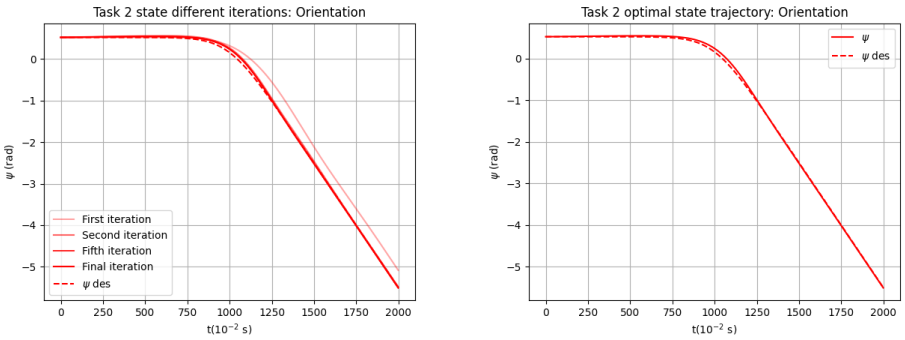


Figure 3.7: Desired smooth position trajectory

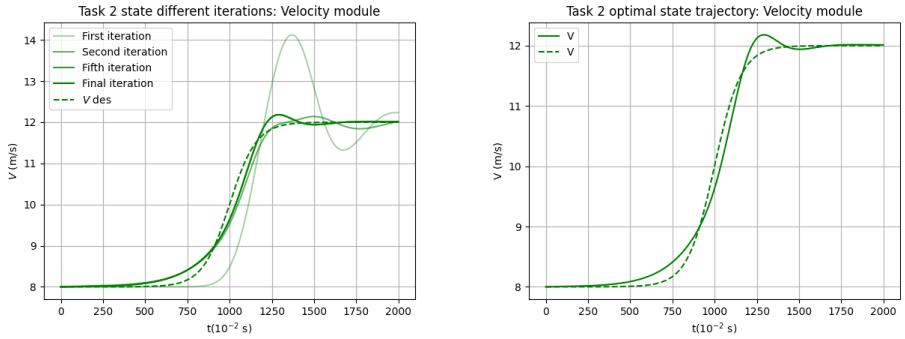


Figure 3.8: Desired smooth position trajectory

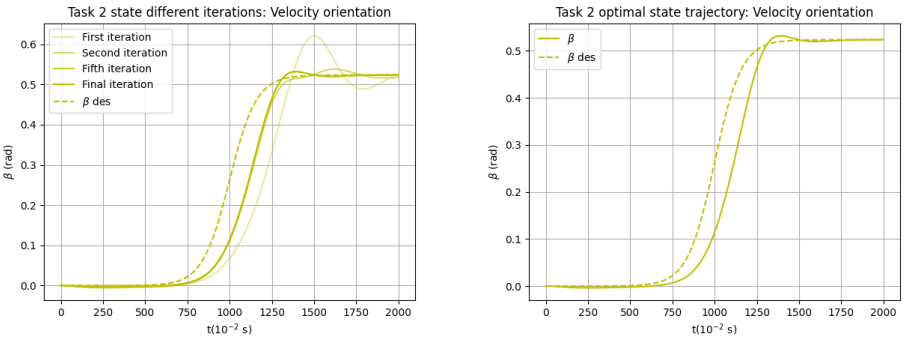


Figure 3.9: Desired smooth position trajectory

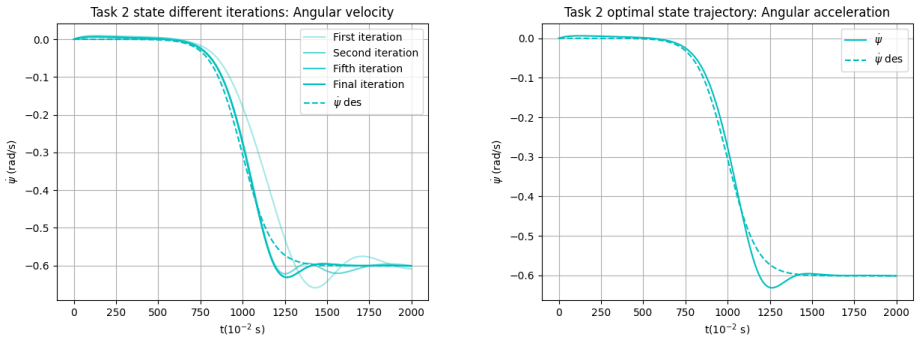


Figure 3.10: Desired smooth position trajectory

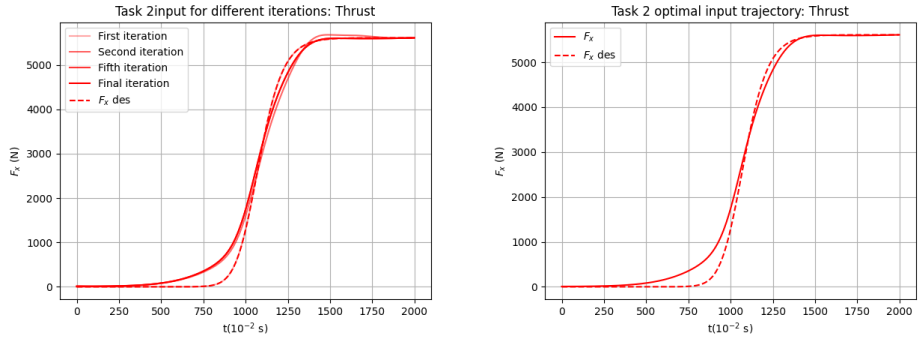


Figure 3.11: Desired smooth position trajectory

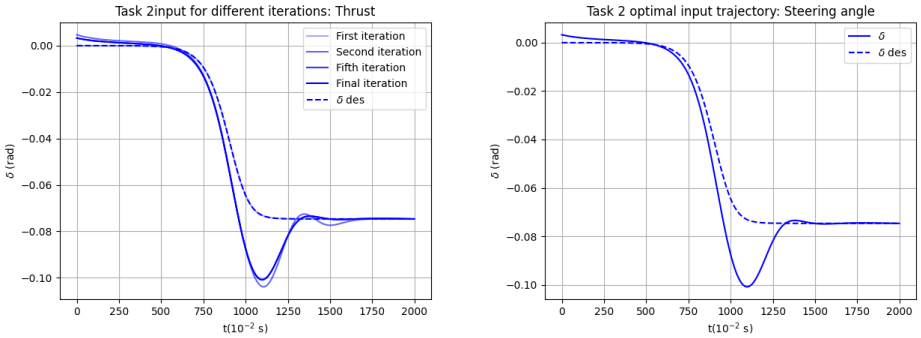


Figure 3.12: Desired smooth position trajectory

Below are some of the Armijo's plots for the secon task. The plot was taken every two iterations.

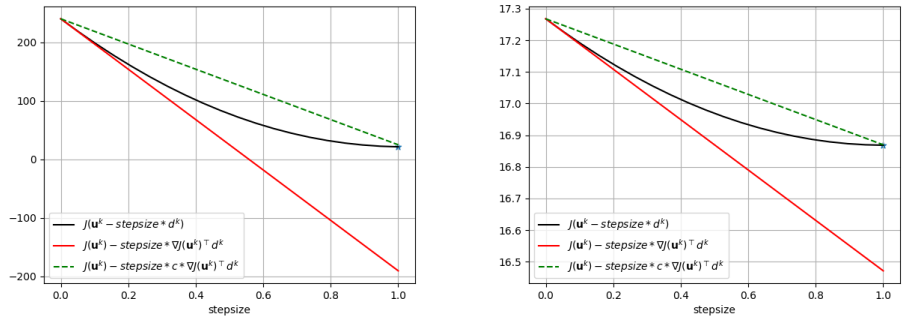


Figure 3.13: Armijo's plot at first and third iteration

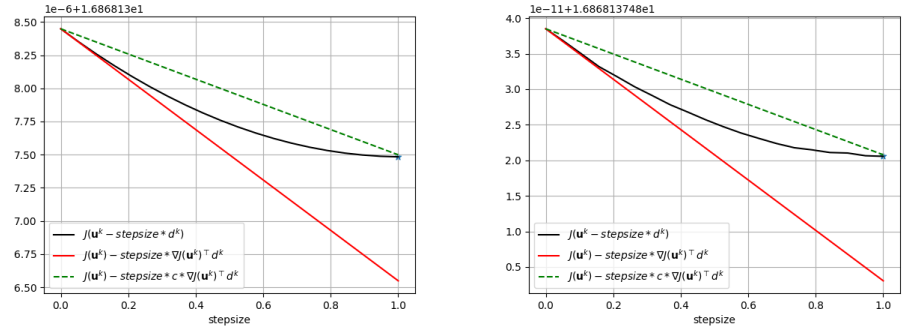


Figure 3.14: Armijo's plot at the fifth and last iteration

Lastly, the plots for the cost and gradient norm over iterations can be found here.

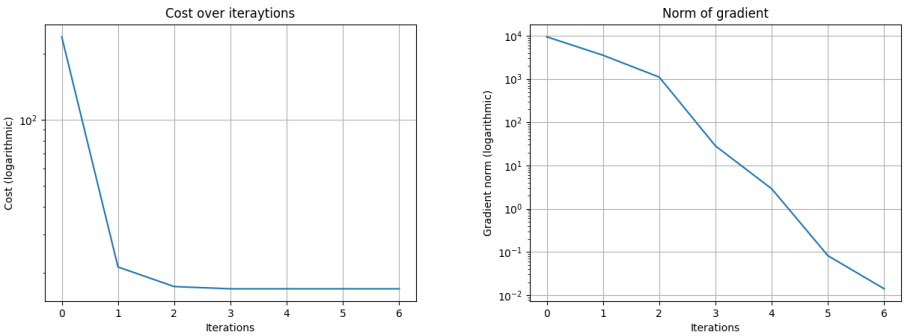


Figure 3.15: Armijo's plot at the fifth and last iteration

3.3 Conclusions

The smooth desired curve leads to a nicer trajectory in terms of overshooting and use of the inputs. The overall cost of the trajectory is $16,87$, which is less then a third of the cost for the step trajectory $(54,58)$. The inputs don't have discontinuities anymore and, as seen in the plots, stay closer to the desired curve (the quasi-static input curve)than in task one. Overall, we can clearly see the advantage in asking for a smooth input state curve rather than a step.

Chapter 4

Task 3 - Trajectory tracking via LQR

In this task the objective is to design a linear quadratic regulator [LQR] able to follow the smooth optimal trajectory generated in the previous task.

4.1 Problem setup

The first step needed to achieve our goal is to linearize the system dynamics, computed in Task 0, about the smooth optimal trajectory (x^{opt}, u^{opt}), obtained in Task 2, in order to obtain A^{opt} and B^{opt}. Then the idea is to use a regulator that will reject disturbances, and will try to follow the optimal trajectory. To do that, we have to solve a Linear quadratic optimization problem, with a quadratic cost function, in which the cost matrices were a degree of freedom, and the linearized dynamics over the optimal trajectory.

4.2 Parameters and results

Running some simulation, we found great results using as matrices Q^{reg} and R^{reg} the same matrices used in the cost function for the Newton's method. To test the performances of our regulator, we will inject in the system some random error (5-10-20% of the maximum value reached during the trajectory for each state) in the initial conditions.
Then we need to solve the LQ optimal control problem to find the feedback gain matrix K_t^{reg} needed to update u bye the equation:

$$u_t = u_t^{opt} + K_t^{reg}(x_t - x_t^{opt})$$
$$x_{t+1} = f_t(x_t, u_t)$$

In the following plots we can see the tracking performances of the LQR algorithm used.

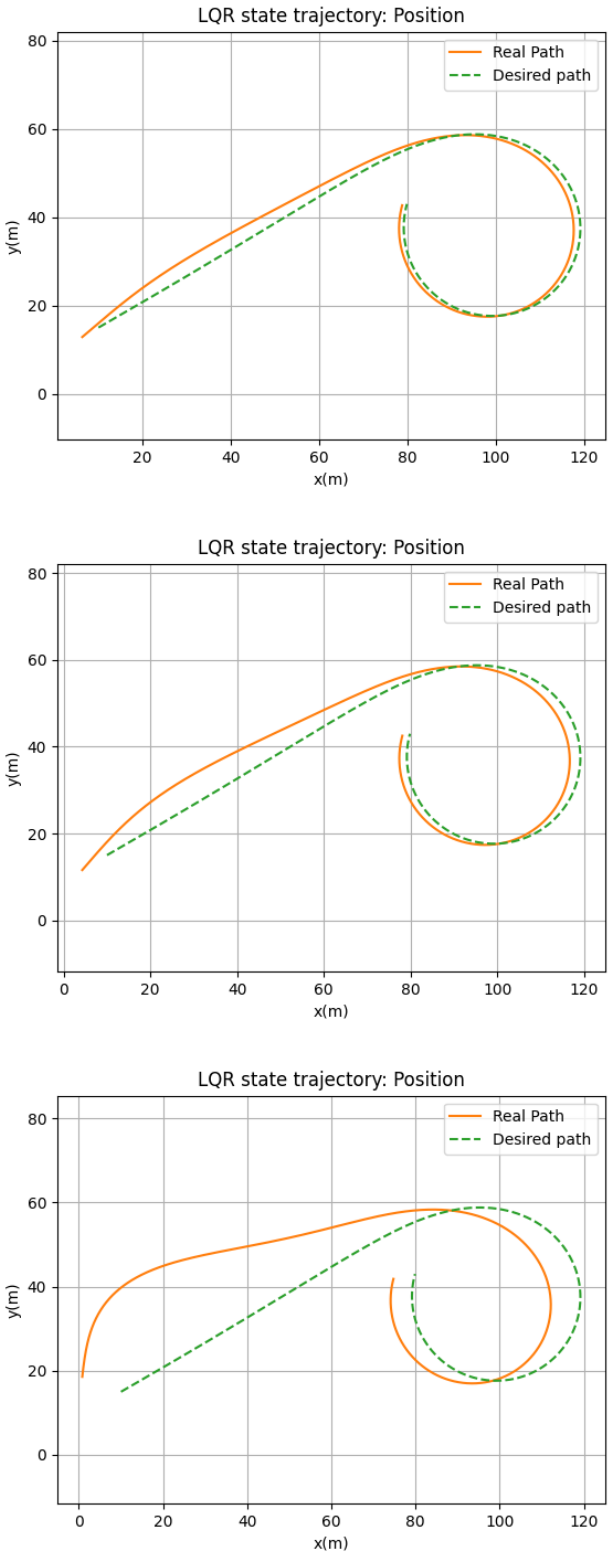


Figure 4.1: Tracked position trajectory with 5-10-20% relative error on x_0

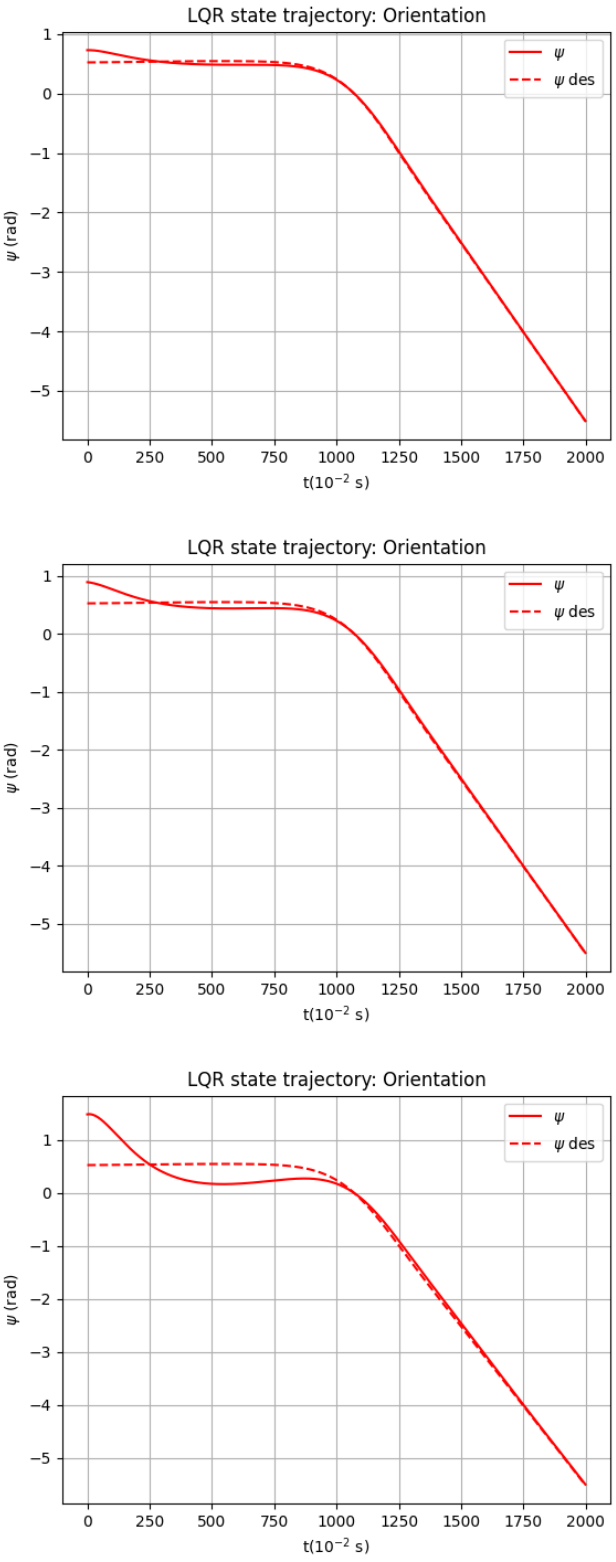


Figure 4.2: Tracked ψ trajectory with 5-10-20% relative error on x_0

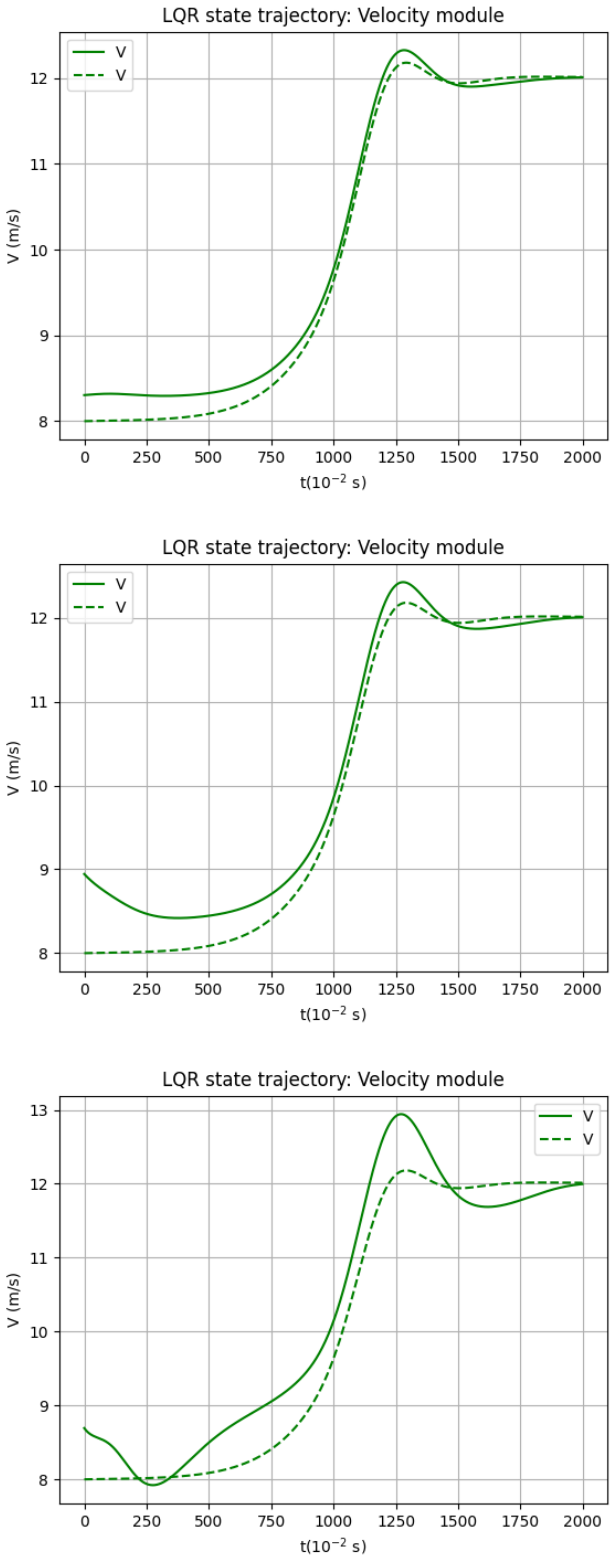


Figure 4.3: Tracked V trajectory with 5-10-20% relative error on x_0

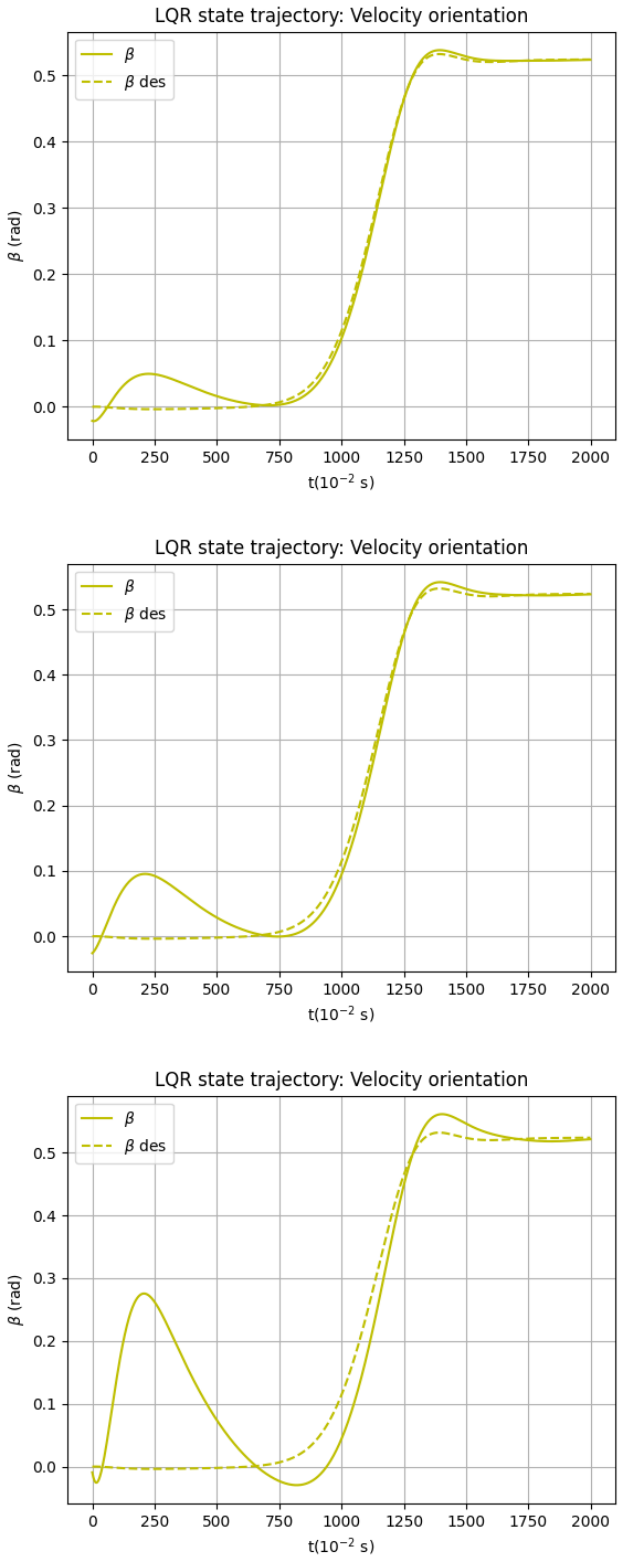


Figure 4.4: Tracked β trajectory with 5-10-20% relative error on x_0

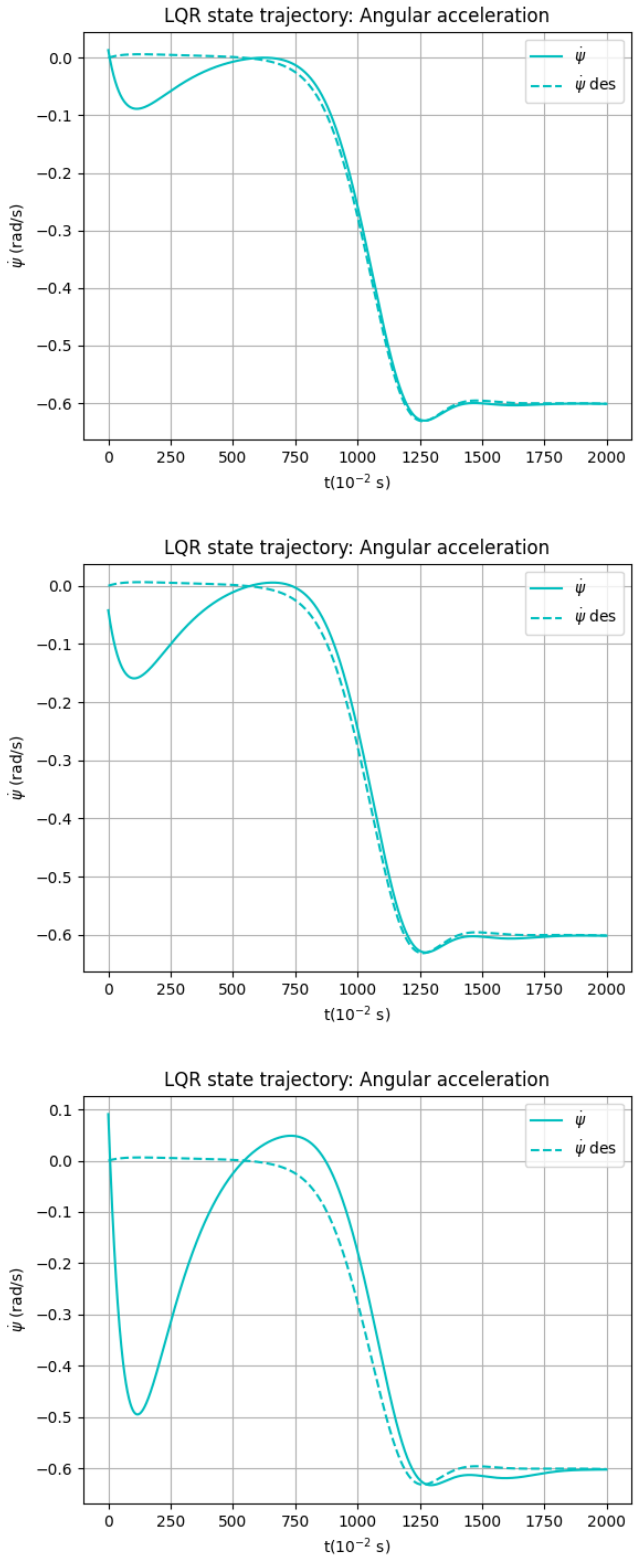


Figure 4.5: Tracked $\dot{\psi}$ trajectory with 5-10-20% relative error on x_0

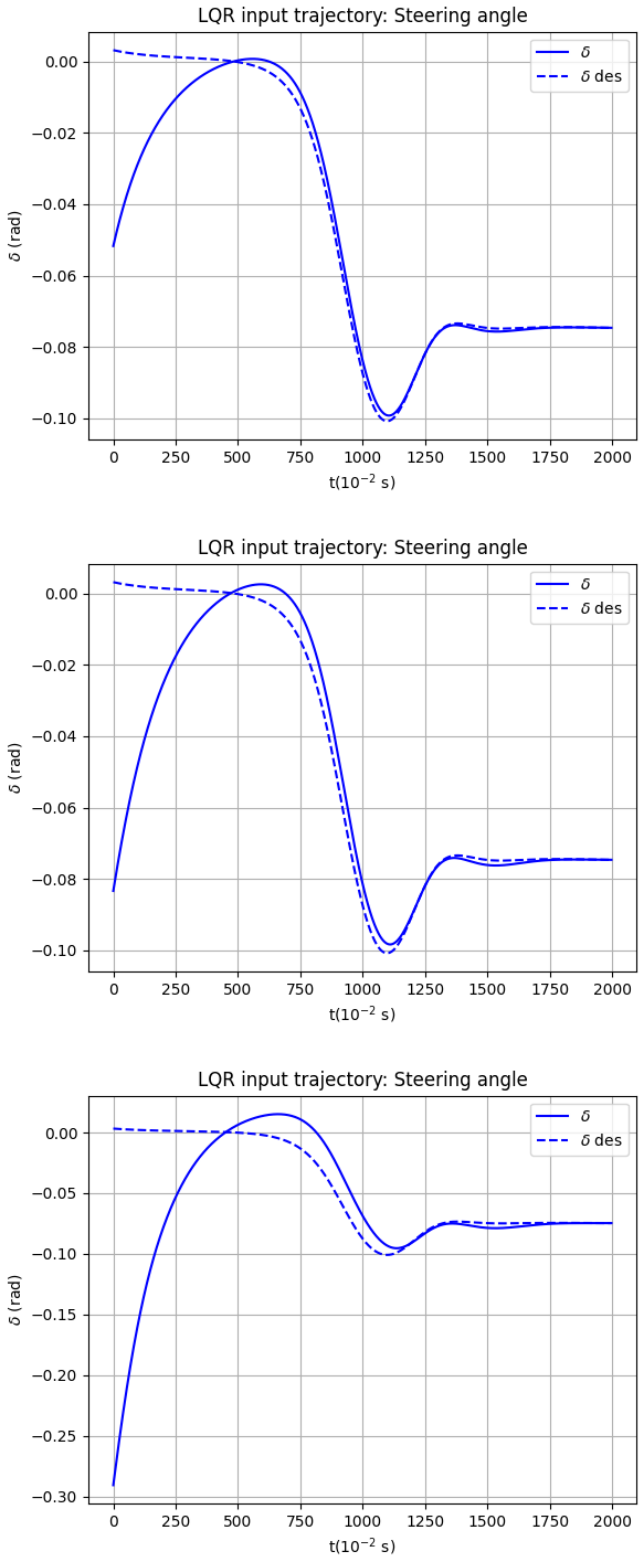


Figure 4.6: Tracked δ trajectory with 5-10-20% relative error on x_0

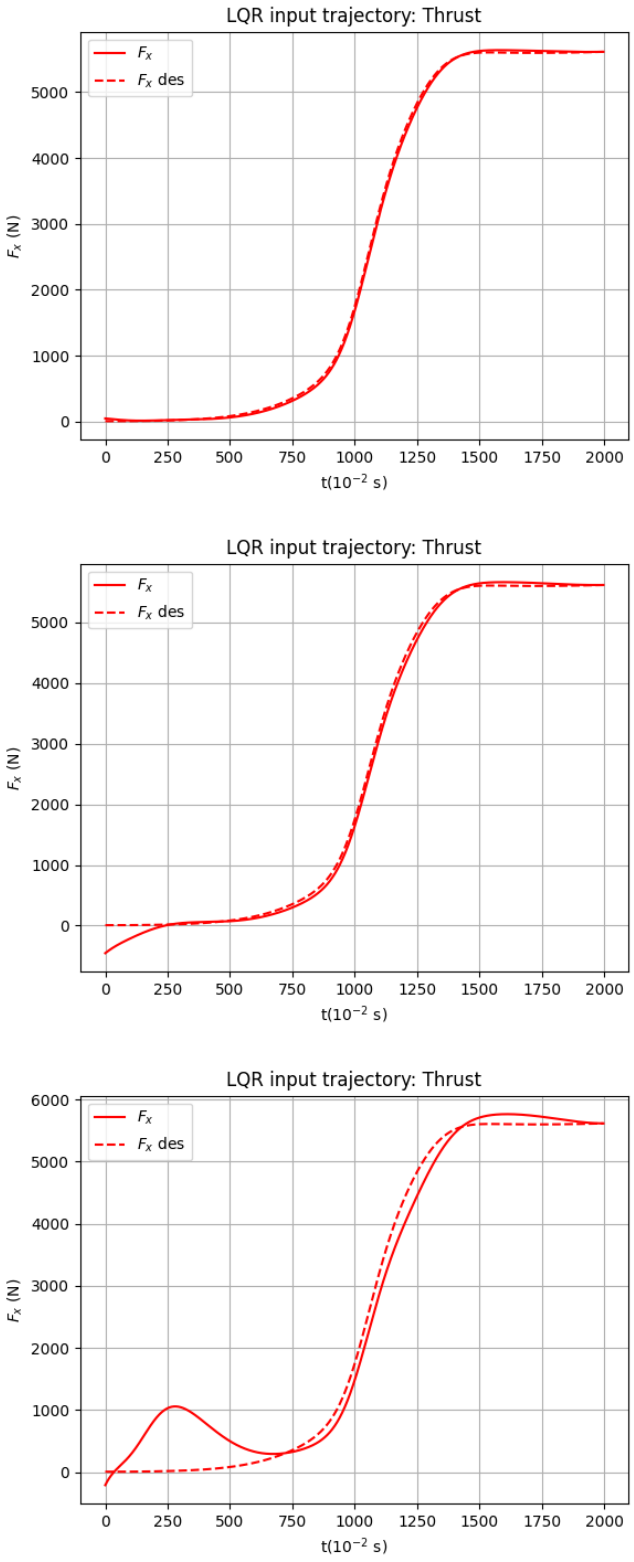


Figure 4.7: Tracked F_x trajectory with 5-10-20% relative error on x_0

4.3 Conclusions

The LQR algorithm proved to be very robust with respect to initial condition error, indeed all the state variables with 5% and 10% error were accurately tracked and only the trajectory with 20% error reached the desired velocity bringing the system in a steady state with some offset in the final position.

Chapter 5

Task 4 - Trajectory tracking via MPC

The goal of this task was to control the system around the optimal trajectory found in task 2 by means of Model Predictive control (hereafter referred as MPC).

5.1 Problem setup

In order to set up an MPC controller, the first step is to linearize around the desired trajectory. This way, we are able to use the `cvxpy` library to solve the problem quickly, by also taking into account possible path constraints on the input or the state. We designed the function `linear_ mpc` specifically to handle this task. At each time instant, we run the optimization problem on a time window of T_{mpc} and use the first input of the sequence. For plotting purposes we keep track of all the inputs we give to the system.

5.2 Parameters

Even if we were able to tune the cost parameters Q_{reg} and R_{reg} as a degree of freedom, after running some experiments we found the same values we used in the optimization problem to be optimal. We implemented a random noise to be added. We tried to evaluate the system evolution for error on one individual component per experiments and the tracking was almost completely matching. However, for lack of space, these results won't be added in this report. We tried to evaluate the evolution of the system with the initial condition x_0 subject to a random error proportional to the maximum aplitude of each component, namely

$$x_{0,i}^{real} = x_{0,i}^{opt} \pm max_t(x_i^{opt}) * p * randn, i \in 1, ..., n \tag{5.1}$$

with p desired percentage, and $randn$ random number such that $-0.5 < randn < 0.5$.

5.3 Results

Below are the plots for the results for p varying from 10 to 40.

5.4 Conclusions

Altough the performances in tracking the velocities are good, even for a small error on the initial condition we reach considerable errors in the steady state values for the position and orientation variables. This is sensitively reduced when the error is on a single initial state component. We tried to address the problem by increasing the costs on the position variables and reducing the costs on the input but that had little to no result. We also tried with different time horizons T_{mpc} and found 10 time instants to be the best trade off between efficiency for control and computational time. Overall, the MPC performances seem to be slightly worse than the LQR ones, but that is explainable via the fact that the LQR gains are computed by taking into account the whole trajectory for the system, while the MPC input takes into account only a small time window.

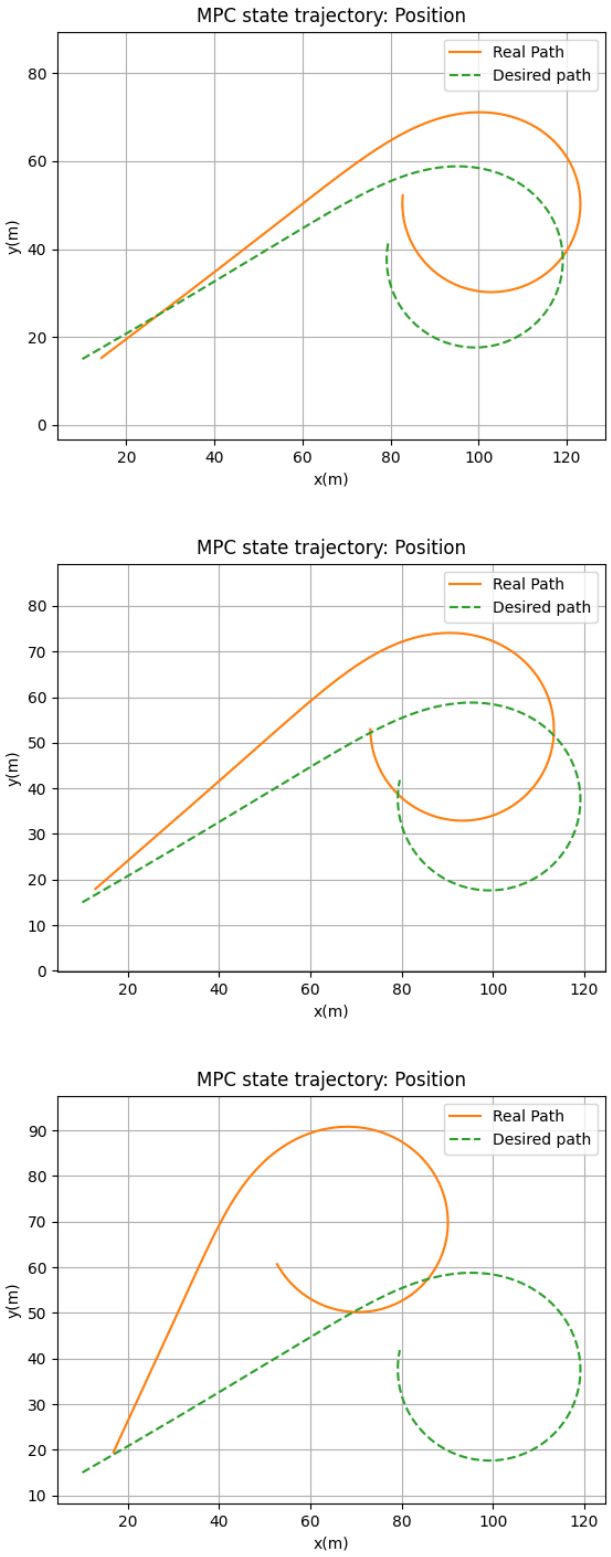


Figure 5.1: Tracked position trajectory with 5-10-20% relative error on x_0

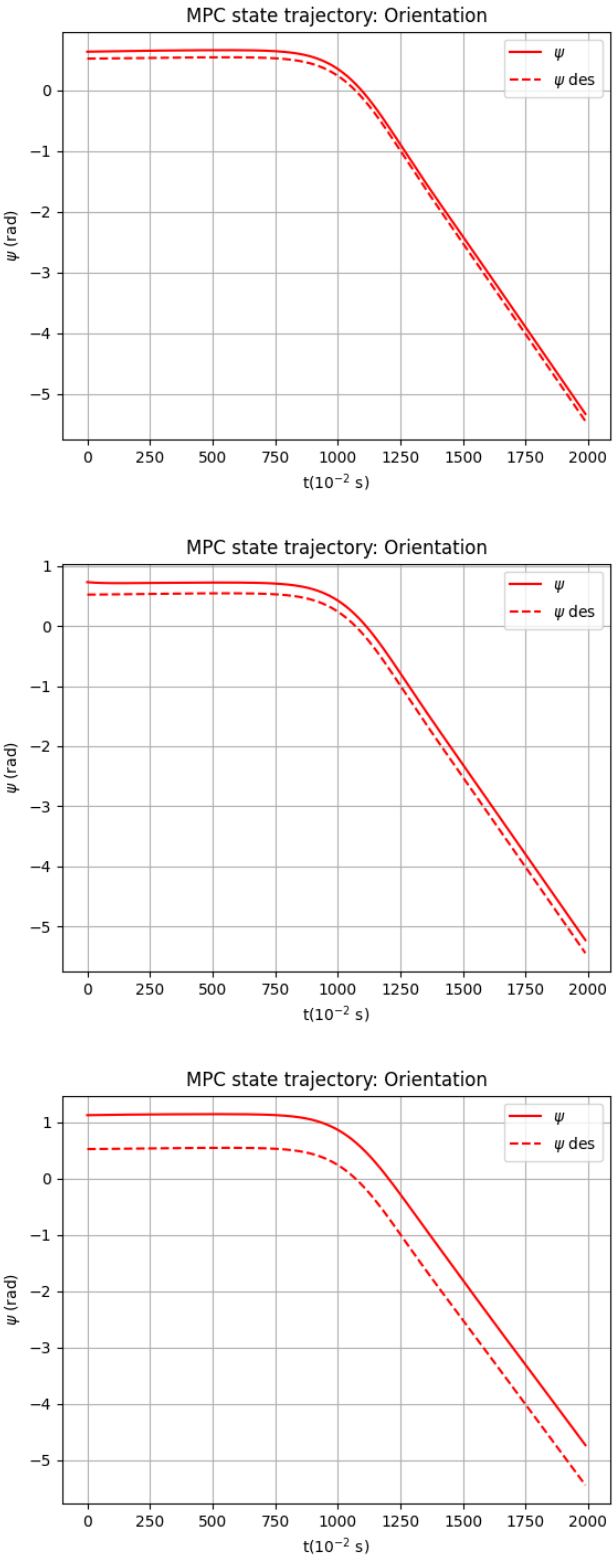


Figure 5.2: Tracked ψ trajectory with 5-10-20% relative error on x_0

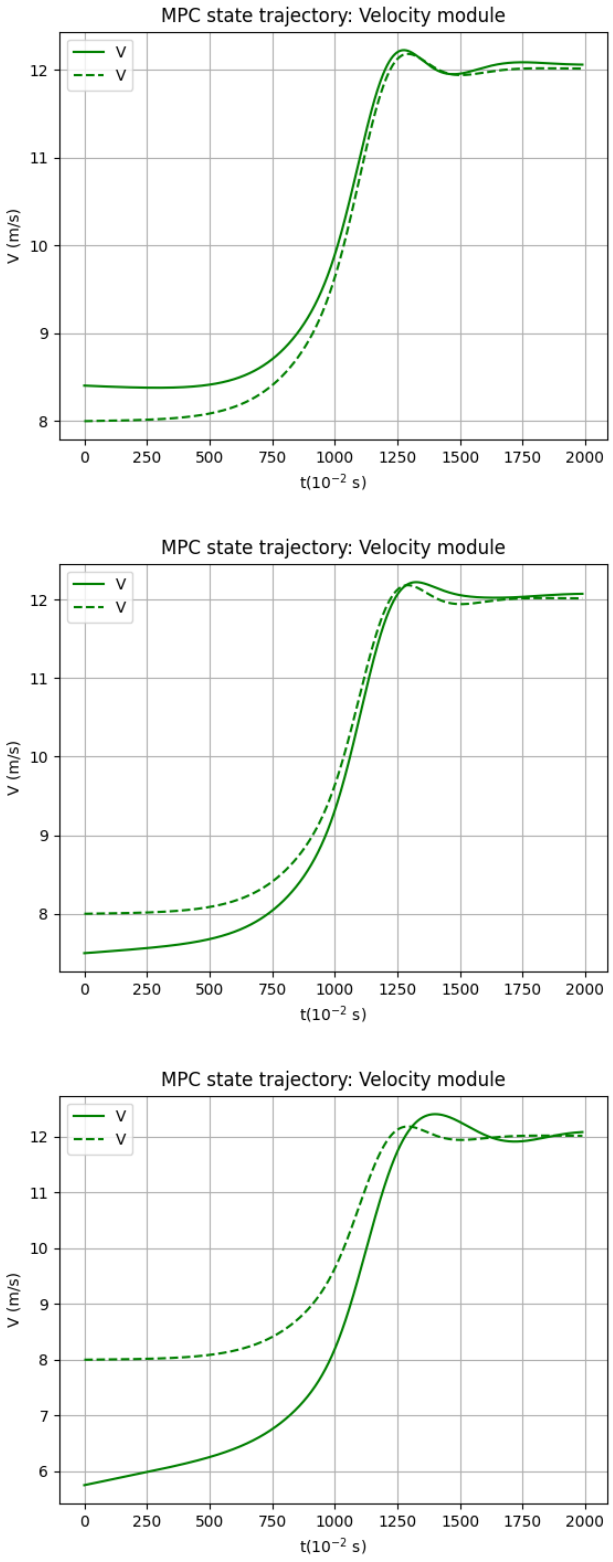


Figure 5.3: Tracked V trajectory with 5-10-20% relative error on x_0

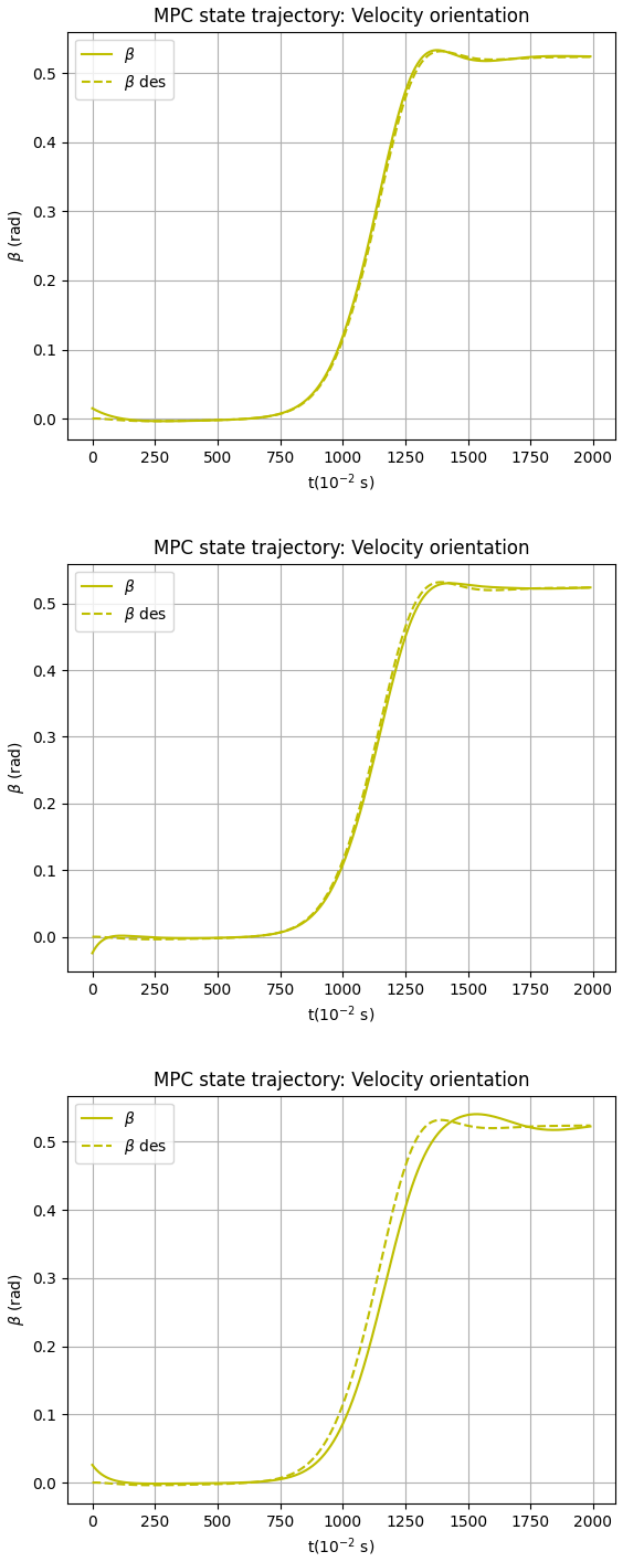


Figure 5.4: Tracked β trajectory with 5-10-20% relative error on x_0

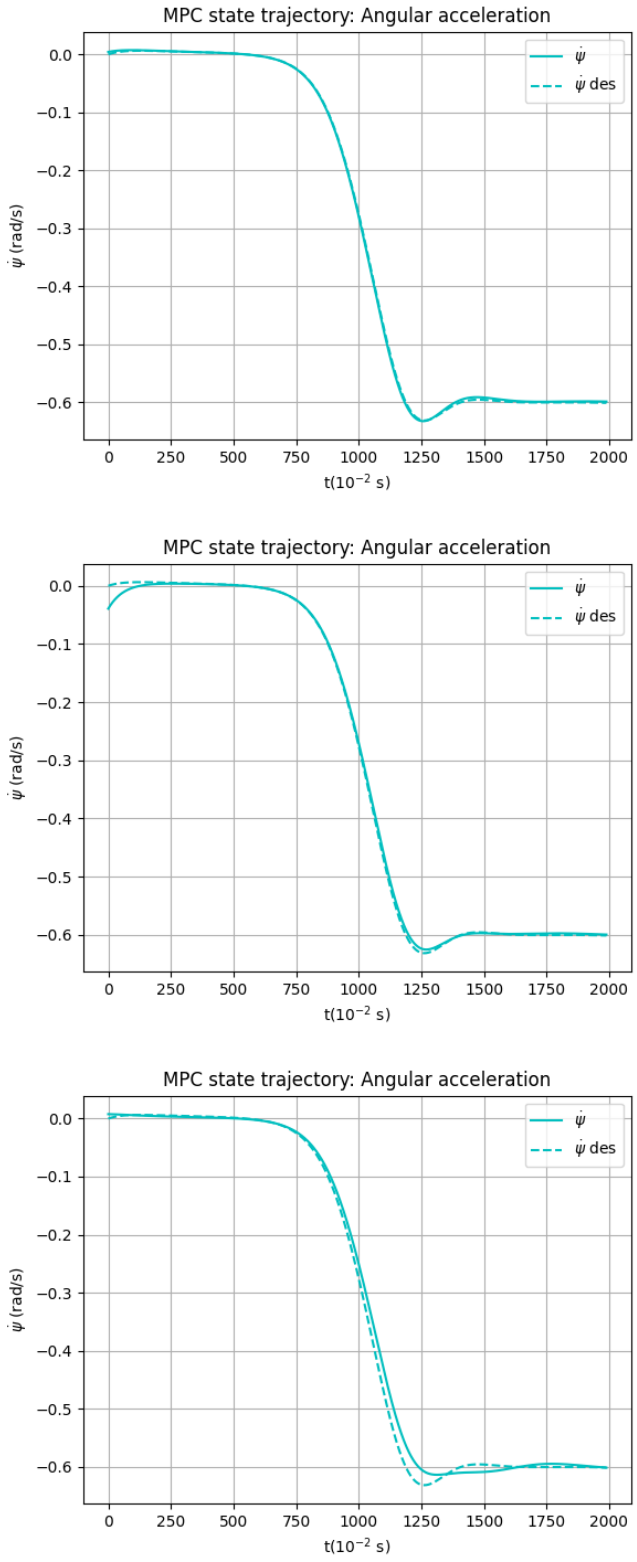


Figure 5.5: Tracked $\dot{\psi}$ trajectory with 5-10-20% relative error on x_0

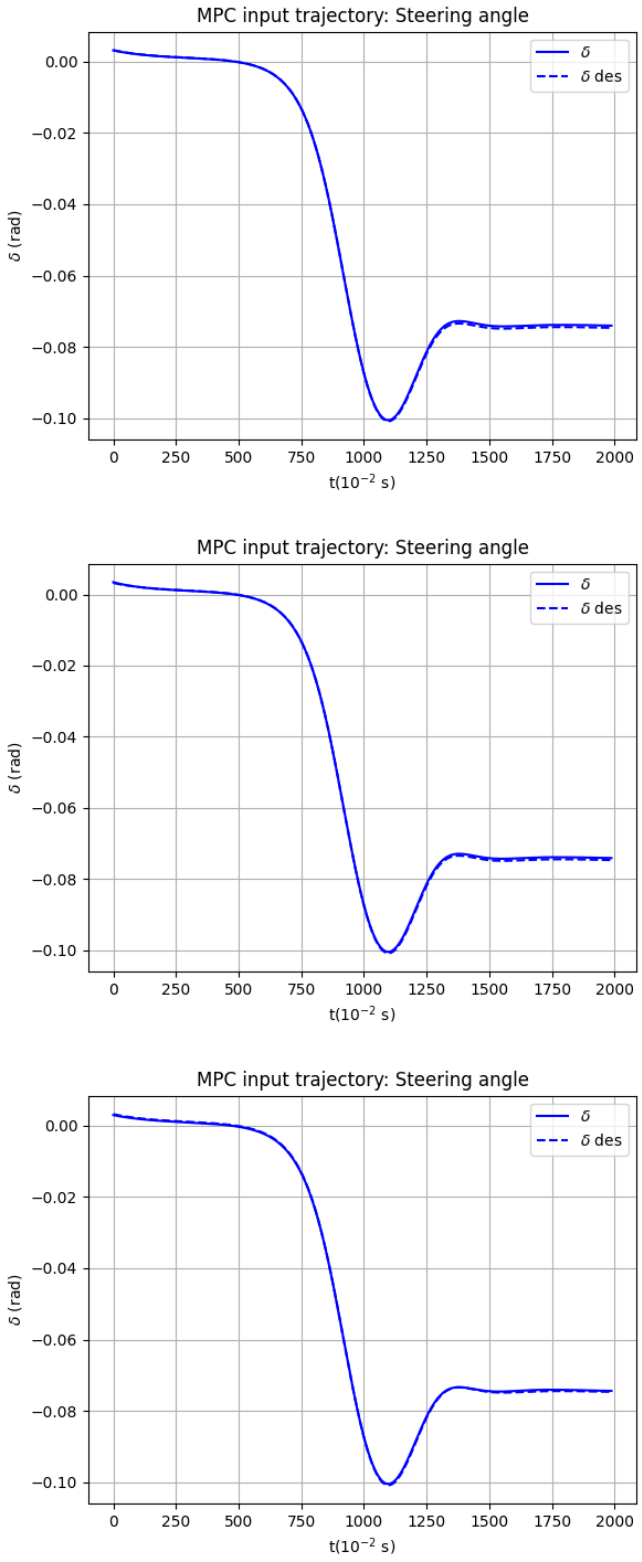


Figure 5.6: Tracked δ trajectory with 5-10-20% relative error on x_0

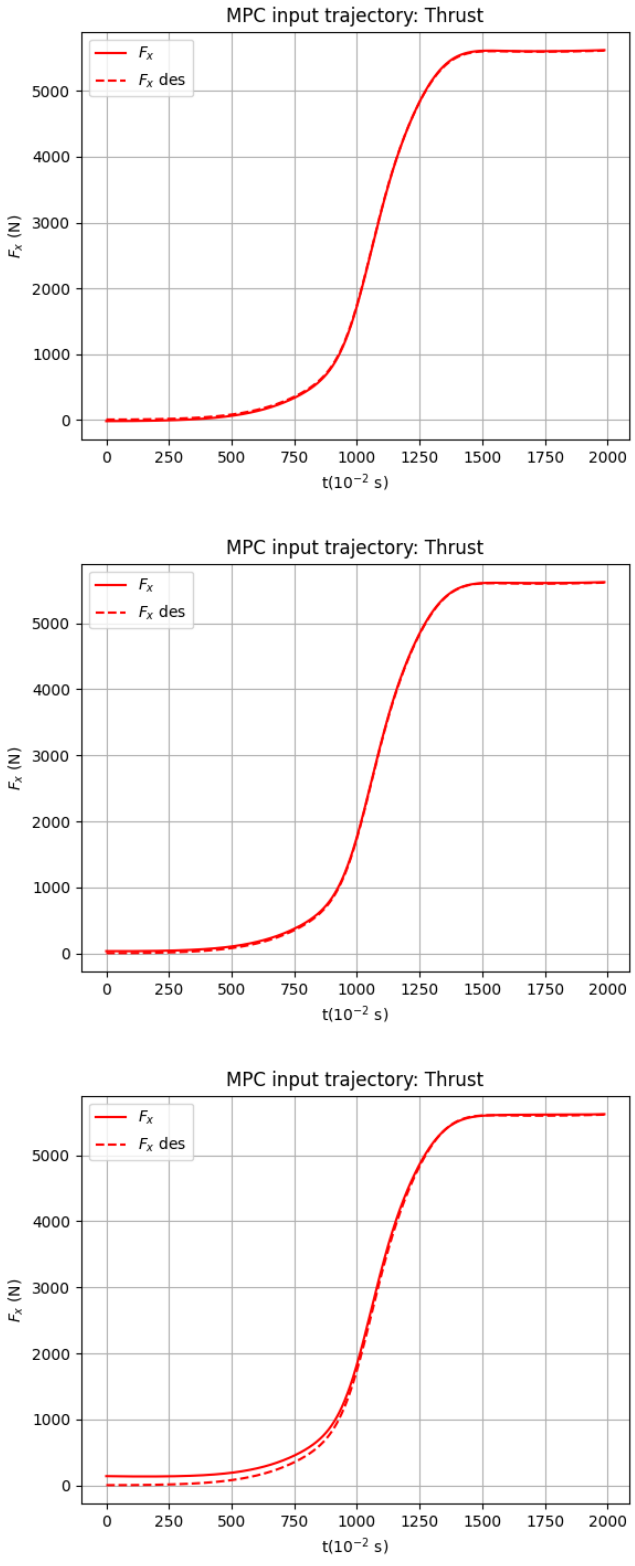
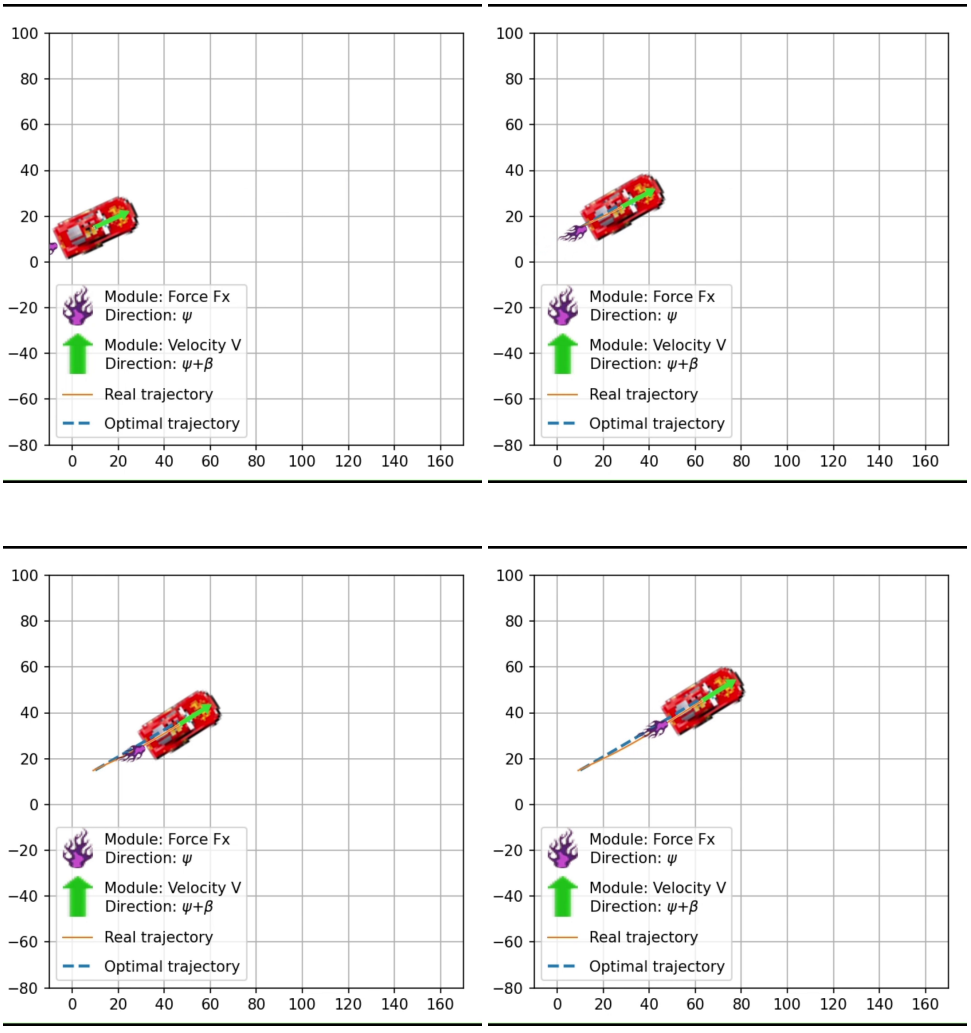


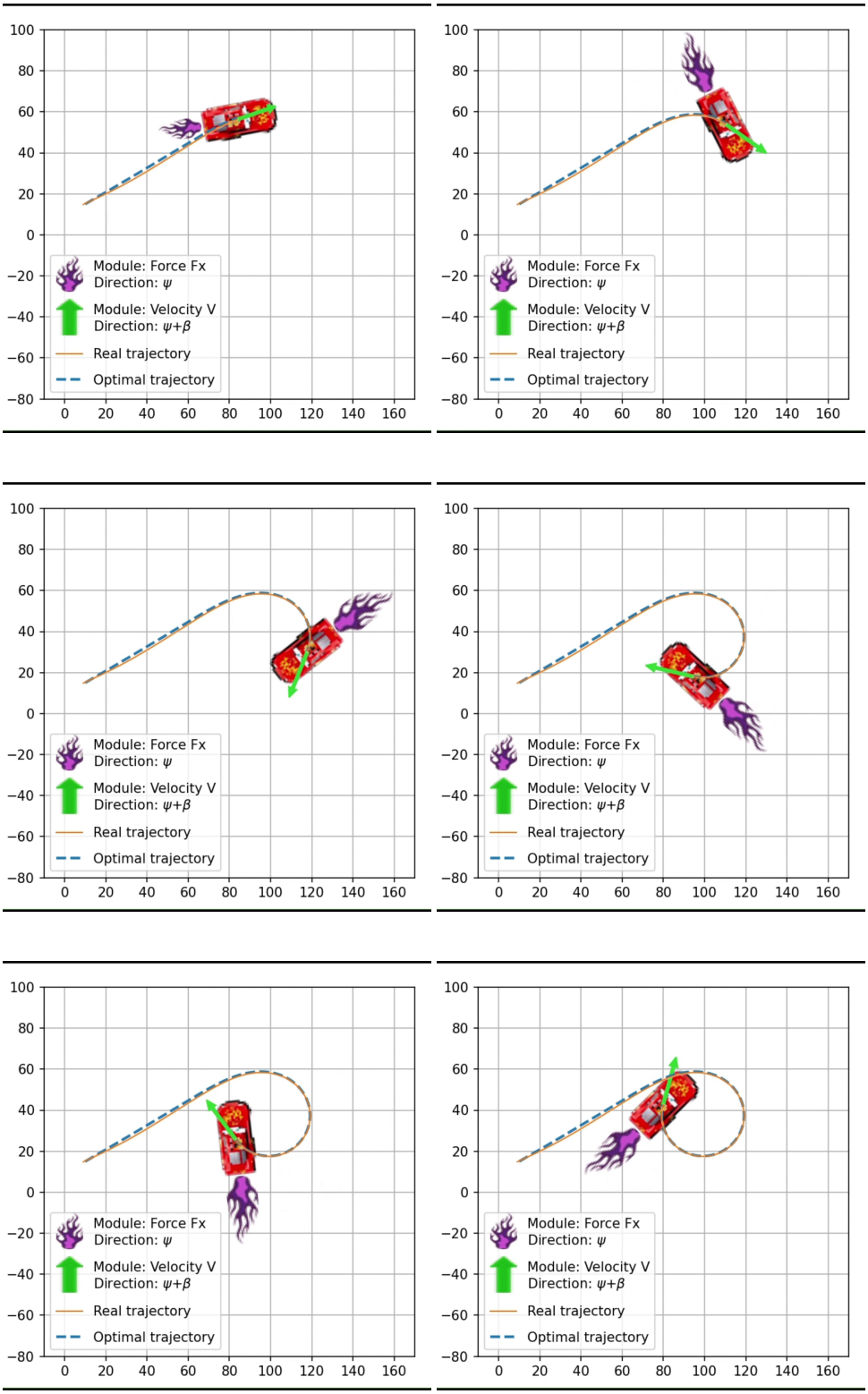
Figure 5.7: Tracked F_x trajectory with 5-10-20% relative error on x_0

Chapter 6

Task 5 - Animation

In this last task we need to animate the tracking of the optimal trajectory with some initial error as obtained in Task 3.
Our best car, Saetta McQueen, is ready to run for us and he is already spitting purple flames to represent the force F_x (purple because `forza Viola`).




Module: Force Fx
Direction: ψ
Module: Velocity V
Direction: ψ+β
Real trajectory
Optimal trajectory

Conclusions

In conclusion this project focused on the use of Python to solve typical optimal control problems.

We modeled the dynamics and the first order derivatives in Task 0 to set-up the problem.
In task 1 we computed two cornering equilibria and created a step desired curve with long tails. We proceeded to use a Newton's approximated algorithm with Armijo's step size selection to compute the optimal trajectory.
In task 2 we moved to a smooth desired curve and found the results to be better in terms of cost and inputs usage.
Tasks 3 and 4 were about controlling with uncertainties around the optimal trajectory and we tested a linear quadratic regulator and a model predictive controller to track the obtained smooth trajectory.
In task 5 we animated the result obtained in task 3.